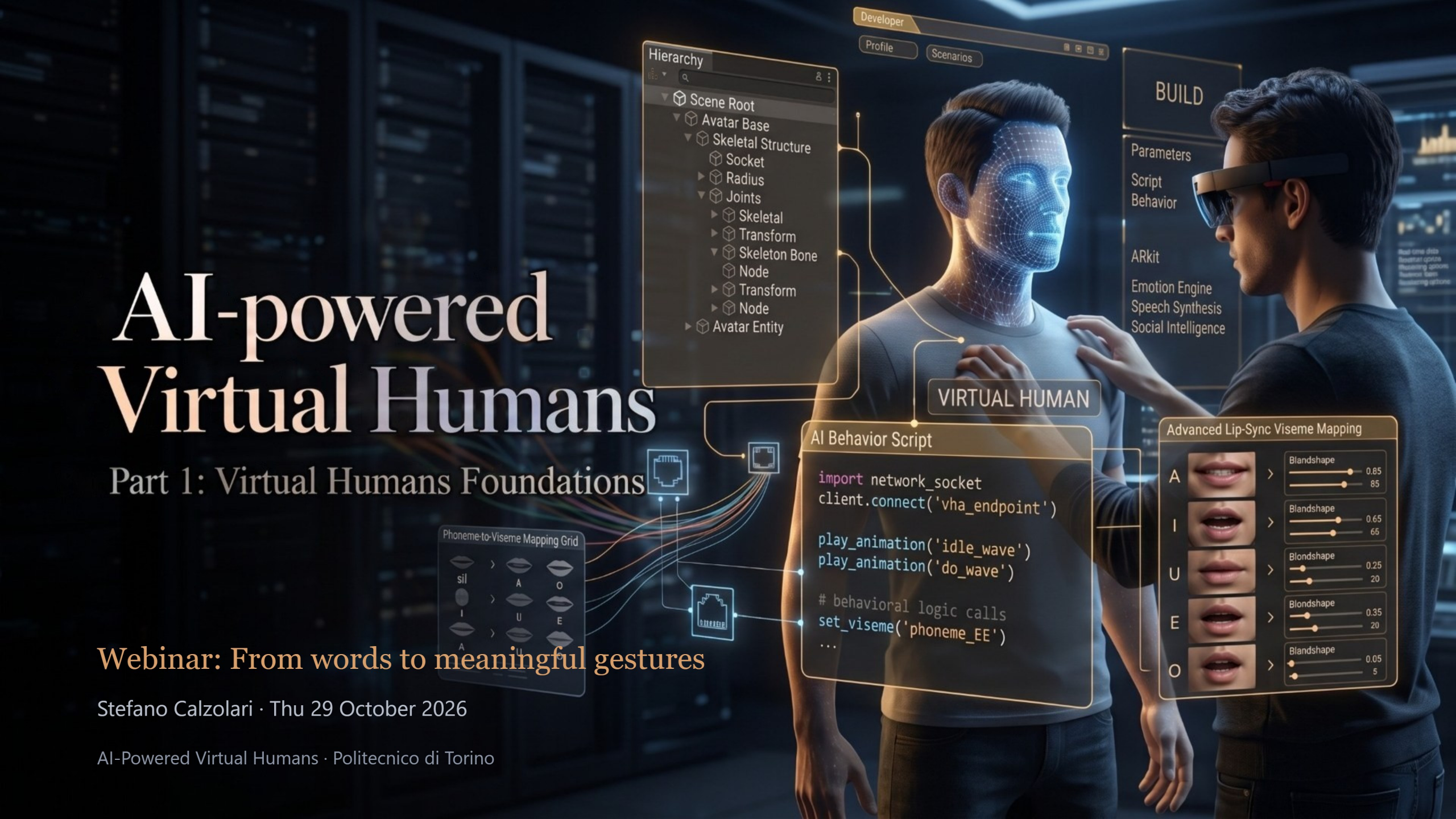


AI-powered Virtual Humans

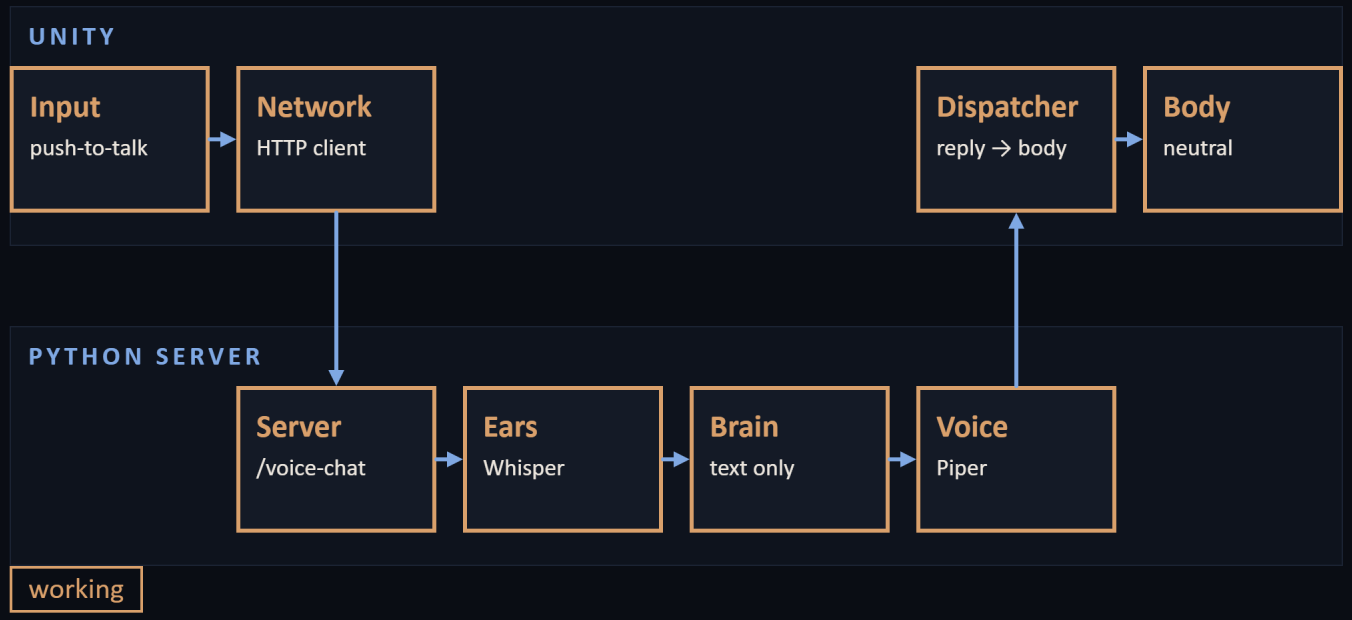
Part 1: Virtual Humans Foundations

Webinar: From words to meaningful gestures

Stefano Calzolari · Thu 29 October 2026

AI-Powered Virtual Humans · Politecnico di Torino

The Architecture



You should have

- ✓ The **voice loop** answers: hold Space, speak
- ✓ The simple **Speak animation** built
- ✓ The **Ollama app**, signed in

Behind? Get on track

1 UNITY

Import the webinar package

Assets → Import Package → Custom Package → **VH_Webinar_Starter** (link: [link]) → Import

2 UNITY

Open the starter scene

Assets/Scenes → **VH_Webinar_Starter**: the Day 2 voice loop, Gesture layer and emotion presets included

3 PYTHON

Bring the Python side up to date

Pull the repo, then in its root folder:

```
python init_webinar.py
```

Day 1–2 gaps solved, today's gaps open. Your old files go to PythonProject/_before_webinar_...

4 CHECK

Start the server and talk

In **PythonProject**, with the .venv active:

```
python -m uvicorn main:app --port 8000
```

Then Play, hold **Space**, speak: Camila answers, body still neutral

Your voice loop already works? Keep your own project and skip this slide.

From words to meaningful gestures

Unity: the body → **FastAPI: the bridge** → **The brain**

By the end of today

You say	The character
“Hello!”	WAVE + HAPPY
Bad news	SAD, no wave
A hard question	THINKING
Something it cannot know	No gesture, NEUTRAL

Smart words, blank body: fixed

The same LLM call must produce three things at once: the **words**, the **gesture** and the **facial expression**, in a format the body can trust.

The system today: what changes



01

Gestures and emotions

How does a gesture play while the character talks?

Who owns which part of the face?

GESTURE LAYER · UPPER BODY

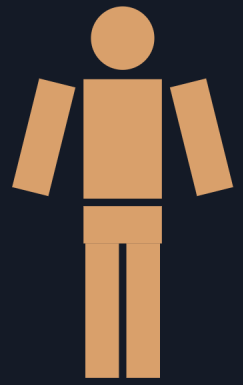
Waving | Thinking

FACE · EMOTIONS

NEUTRAL | HAPPY | SAD

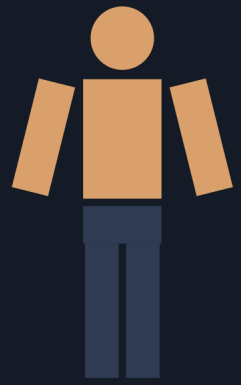
Why a separate gesture layer

One layer



Wave replaces the whole pose: talking stops

Two layers



Wave moves only the upper body

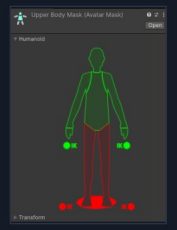
Built before today

Piece	Setting
Gestures layer	Upper body, Mask = UpperBodyMask, with the speaking beats
Empty state	Default: does nothing
Empty → Waving / Thinking	Condition = the trigger
Any State → Empty	Has Exit Time on, Speaking = false

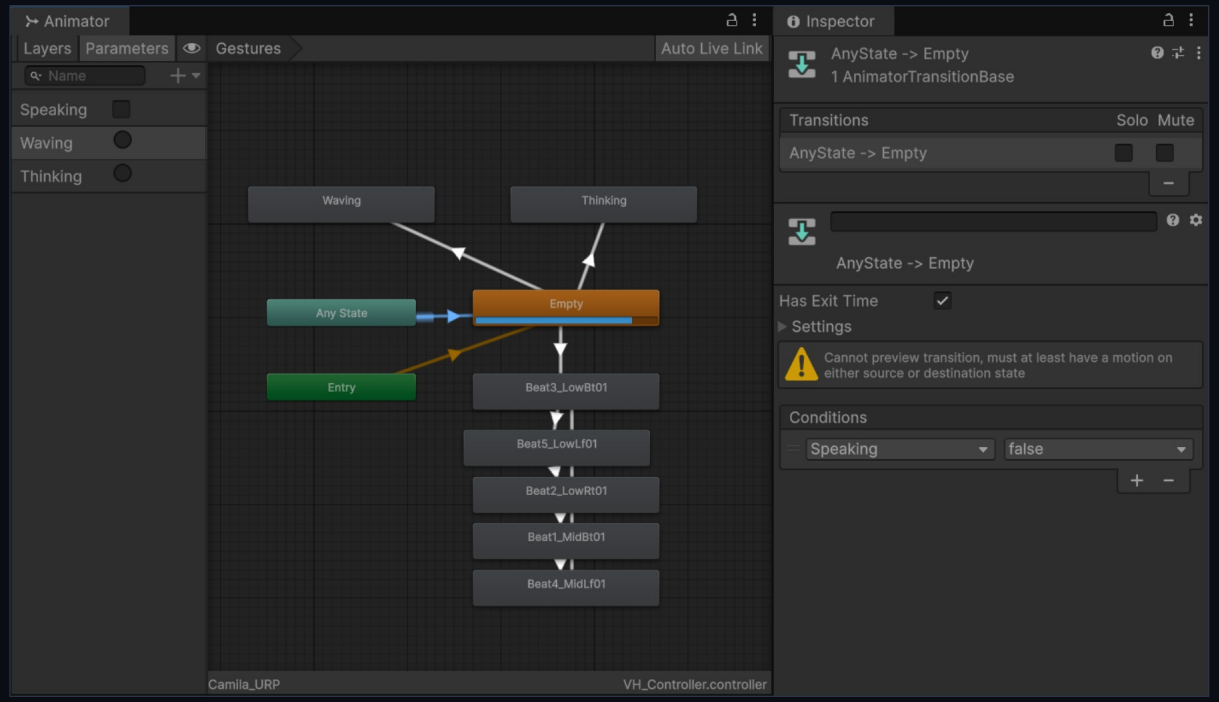
Avatar Mask = which bones a layer may touch

The Gesture layer, step by step

1. Open **VH_Controller** → Parameters: next to **Speaking** (Bool) add two **Triggers**: **Waving**, **Thinking**
2. Select the **Gestures** layer: the upper-body layer that already holds **Empty** and the speaking beats
3. Drag the waving and thinking clips in → name the states **Waving**, **Thinking**
4. Right-click **Empty** → Make Transition → **Waving**: Condition **Waving**. Same for **Thinking**
5. Right-click **Any State** → Make Transition → **Empty**: Has Exit Time **on**, Condition **Speaking = false**: the way back



UpperBodyMask: same mask and settings as the **Speaking** layer from Day 1 (IK Pass off). Green = the layer may move it.



LIVE-7: PlayGesture with a whitelist

CharacterBehaviorManager.Gestures.cs

```
// CharacterBehaviorManager.Gestures.cs (partial, provided)
static readonly Dictionary<string, int> Gestures = new() {
    { "WAVE",     Animator.StringToHash("Waving") },
    { "THINKING", Animator.StringToHash("Thinking") } };

public void PlayGesture(string gestureId)
{
    // ===== LIVE-7 · Body ▸ Gesture layer =====
    if (string.IsNullOrWhiteSpace(gestureId)) return;
    string id = gestureId.Trim().ToUpperInvariant();
    if (id == "NONE") return;
    if (!Gestures.TryGetValue(id, out int trigger))
    { Debug.LogWarning($"Unknown gesture '{gestureId}'"); return; }
    animator.SetTrigger(trigger);
}
```

Public ids for the LLM, trigger names inside

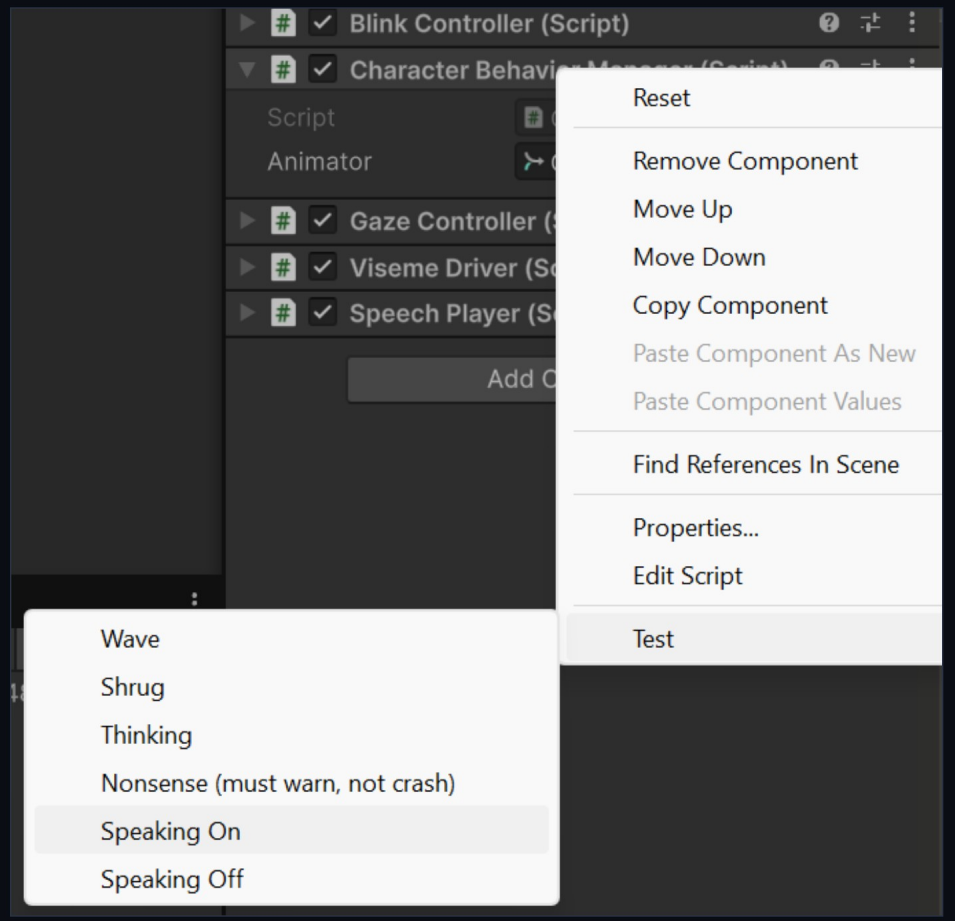
LLM output is untrusted input

Unknown: a warning, never a crash

Try it: gestures from the context menu

1. **Play**
2. Select **Camila_URP** (root) → right-click the **Character Behavior Manager** header → **Test** → **Wave**, **Thinking**
3. **Speaking On**, then **Wave**: the arms wave, the talking/idle resumes after

> Nothing moves? The Animator trigger names must be exactly **Waving**, **Thinking**



Three facial expressions



NEUTRAL



HAPPY



SAD

Who owns which blendshape?

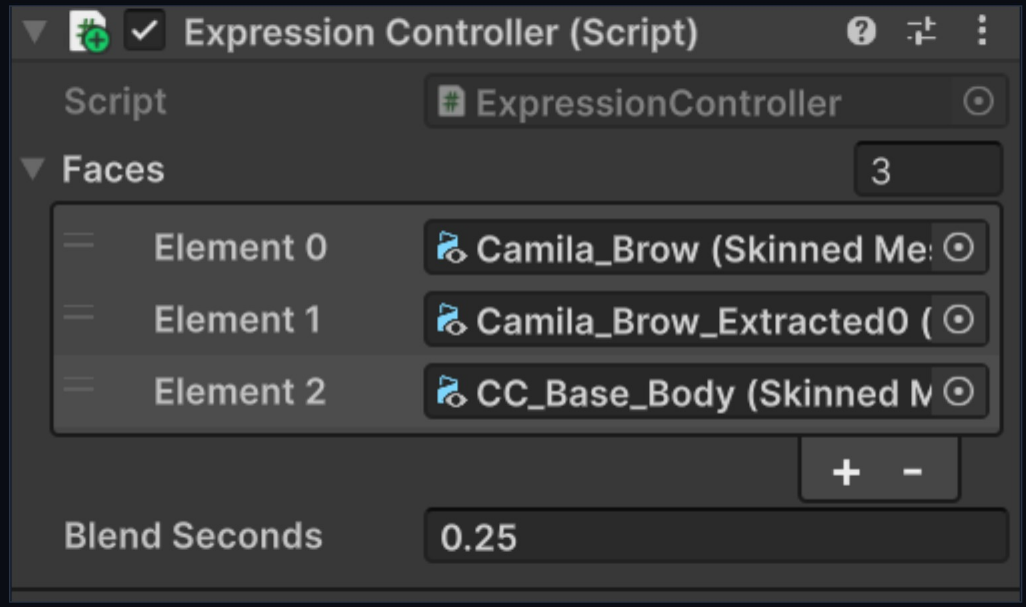
Mouth jaw, lips: the viseme (V_) shapes	`lip-sync`
Eyelids the eye-closing shapes	`blink`
Brows and eyes brows, cheeks, eyes, the smile corners	`emotion`

One owner per shape. Two scripts writing the same blendshape in the same frame: the last one wins.

Add the ExpressionController

1. Select **Camila_URP** (root) → Add Component → **ExpressionController**
2. **Faces**: size **3** → drag **Camila_Brow**, **Camila_Brow_Extracted0**, **CC_Base_Body** from Camila_URP's children
3. Blend Seconds: keep **0.25**
4. Play → right-click **Expression Controller** → **Emotion** → **Happy**, **Sad**, **Neutral**

No presets in the Inspector: **HAPPY** and **SAD** are written in the script



PRESETS · ExpressionController.cs

HAPPY Cheek_Raise 70 · Brow_Raise_Inner 25 · Eye_Squint 35 · Mouth_Smile_L 55 · Mouth_Smile_R 55

SAD Brow_Raise_Inner 90 · Brow_Compress 30 · Look_Down 30

ExpressionController 1/2: SetEmotion

```
[SerializeField] SkinnedMeshRenderer[] faces;
static readonly List<EmotionPreset> Presets = new List<EmotionPreset> {
    Preset("HAPPY", ("Cheek_Raise", 70), ("Brow_Raise_Inner", 25),
        ("Eye_Squint", 35), ("Mouth_Smile_L", 55), ("Mouth_Smile_R", 55)),
    Preset("SAD",   ("Brow_Raise_Inner", 90), ("Brow_Compress", 30),
        ("Look_Down", 30)) };

[SerializeField] float blendSeconds = 0.25f;

void Awake()  // resolve every preset shape on every face mesh

public void SetEmotion(string emotionId)
{
    Array.Clear(target, 0, target.Length);       // neutral
    if (string.IsNullOrWhiteSpace(emotionId)) return;
    string id = emotionId.Trim().ToUpperInvariant();
    if (!resolved.TryGetValue(id, out var shapes)) return;
    foreach (var (slot, weight) in shapes) target[slot] = weight;
}
```

Same whitelist idea: unknown → ignored

Clear first: back to **neutral**

Presets: **brows, cheeks, eyes** + smile corners

ExpressionController 2/2: blending over time

```
void Update()          // ease each weight toward its target
{
    float speed = 100f / Mathf.Max(0.01f, blendSeconds);
    for (int i = 0; i < current.Length; i++)
    {
        if (Mathf.Approximately(current[i], target[i])) continue;
        current[i] = Mathf.MoveTowards(current[i], target[i],
                                       speed * Time.deltaTime);
        slots[i].Set(current[i]);      // on every face mesh
    }
}
```

A little **every frame**: no snapping

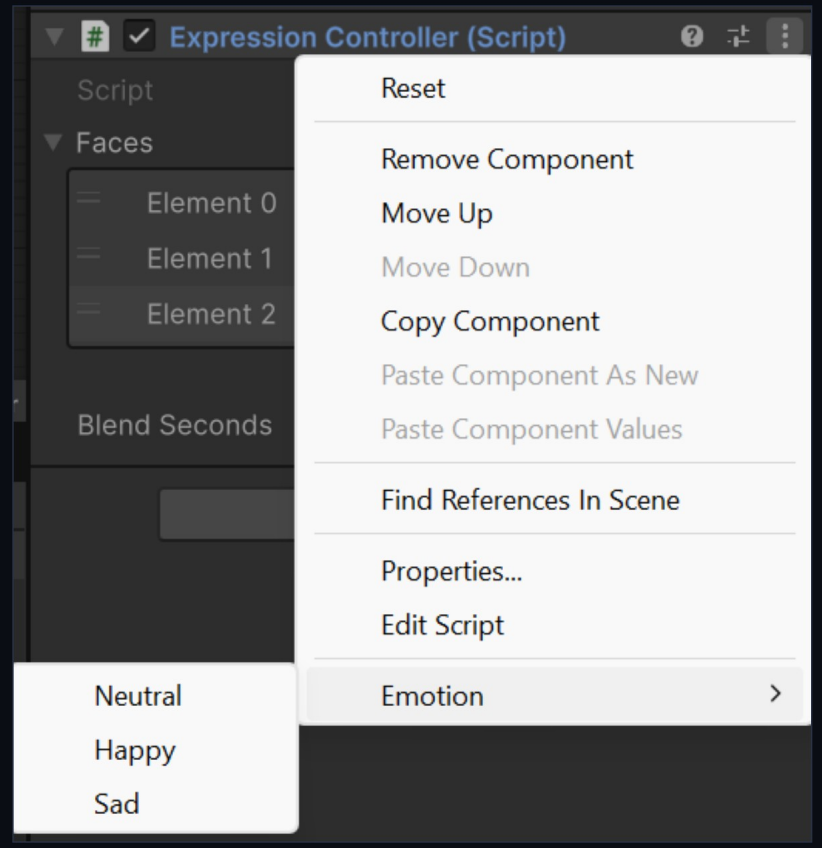
Writes only shapes that change

Test with the sound **off**: can you tell **HAPPY** and **SAD** apart?

Try it: emotions from the context menu

1. **Play**
2. Select **Camila_URP** (root) → ⋮ on the **Expression Controller** header → **Emotion** → **Happy**, **Sad**, **Neutral**
3. **Happy**: cheeks up and a light smile · **Sad**: inner brows up, eyes down · **Neutral**: the face eases back in 0.25 s
4. Play the lip-sync test clip, then **Happy**: the lips talk and smile together, no flicker

> Nothing moves? Check the three **Faces**, and the Console for “Blendshape containing … not found”



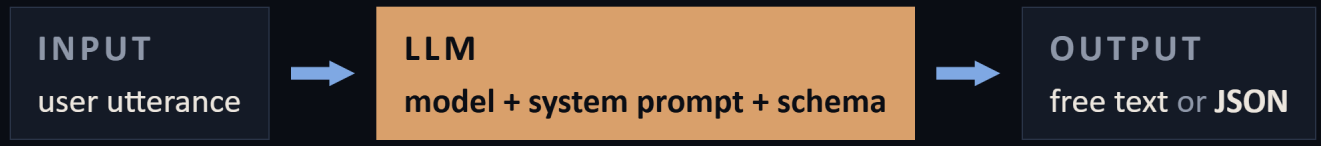
02

Structured LLM output

How can we use the LLM as a decisional engine?

```
{
  "speech_text": "Hello! Nice to meet you.",
  "action": "WAVE",
  "emotion": "HAPPY"
}
```

What is structured output?



FREE TEXT OUTPUT

```
Hello! Nice to meet you.
```

Words only: the body has to **guess** what to do

STRUCTURED OUTPUT (JSON)

```
{
  "speech_text": "Hello! Nice to meet you.",
  "action": "WAVE",
  "emotion": "HAPPY"
}
```

Fixed fields, allowed values: code can **read** every part

1 A JSON schema

BrainOutput in Pydantic: the fields and the values each one may take

2 Sent with the LLM call

format= the JSON schema: the model can only produce that shape

3 Checked on arrival

Pydantic validates it; anything broken → the **fallback**, never a crash

TURN-5: action and emotion in the contract

schemas.py

```python
Action = Literal["NONE", "WAVE",
                 "THINKING"]
Emotion = Literal["NEUTRAL", "HAPPY", "SAD"]

class BrainOutput(BaseModel):
    model_config = ConfigDict(extra="forbid")
    speech_text: str
      # ===== TURN-5 =====
      # + action  (an Action,  default "NONE")
      # + emotion (an Emotion, default "NEUTRAL")
```

AgentNetworkClient.cs

```
[Serializable] public class ChatResponse
{
    public string speech_text;
    public string audio_url;
      public string action, emotion;  // already there
}
```

An example of reply

```
{
  "speech_text": "Hello! Nice to meet you.",
  "audio_url": "/static/audio/3f2a….wav",
  "action": "WAVE",
  "emotion": "HAPPY"
}
```

TURN-5: solution

schemas.py

```python
Action = Literal["NONE", "WAVE", "THINKING"]
Emotion = Literal["NEUTRAL", "HAPPY", "SAD"]

class BrainOutput(BaseModel):
    model_config = ConfigDict(extra="forbid")
    speech_text: str
    # ===== TURN-5 =====
      action: Action = "NONE"
      emotion: Emotion = "NEUTRAL"
```

TURN-6: the action inventory

```
SYSTEM_PROMPT = """You are Camila, ...   # persona and style: unchanged

OUTPUT
Reply with ONE JSON object and nothing else:
{"speech_text": "<what you say>", "action": "<ACTION>", "emotion": "<EMOTION>"}

ACTION must be exactly one of: NONE, WAVE, THINKING
  <TURN-6: one line per action saying WHEN to use it>

EMOTION must be exactly one of: NEUTRAL, HAPPY, SAD
Bad news or sad topics: SAD, usually with NONE.
Never use an action or emotion that is not in the lists."""
```

Write **when**, not what: “WAVE: greetings and goodbyes”. The model already knows what a wave is.

TURN-6: solution brain.py

```
SYSTEM_PROMPT = """You are Camila,...   # persona and style: unchanged

OUTPUT
Reply with ONE JSON object and nothing else:
{"speech_text": "<what you say>", "action": "<ACTION>", "emotion": "<EMOTION>"}

ACTION must be exactly one of: NONE, WAVE, THINKING
  - WAVE: greetings and goodbyes
  - THINKING: the question is hard or open-ended
  - NONE: otherwise

EMOTION must be exactly one of: NEUTRAL, HAPPY, SAD
Bad news or sad topics: SAD, usually with NONE.
Never use an action or emotion that is not in the lists."""
```

LIVE-8: the structured LLM call `brain.py`

```python
def think_structured(user_text: str) -> BrainOutput:
    # ===== LIVE-8 · Brain ▸ LLM ▸ Contract =====
    try:
        resp = _client.chat(model=_MODEL, messages=[
                {"role": "system", "content": SYSTEM_PROMPT},
                {"role": "user", "content": user_text}],
            format=BrainOutput.model_json_schema(),  # the contract
            options={"temperature": _TEMPERATURE})
        return _parse(resp.message.content)          # don't trust
    except (ValidationError, ValueError) as e:
        print("Brain: invalid JSON:", e); return _fallback()
    except Exception as e:
        print("Brain: LLM call failed:", e); return _fallback()

def think(user_text):                # THE SEAM: switch it
    return think_structured(user_text)
```

format: the JSON schema of BrainOutput

_parse + Pydantic: validated, always

Bad JSON or no quota → **NONE + SAD**, never a crash

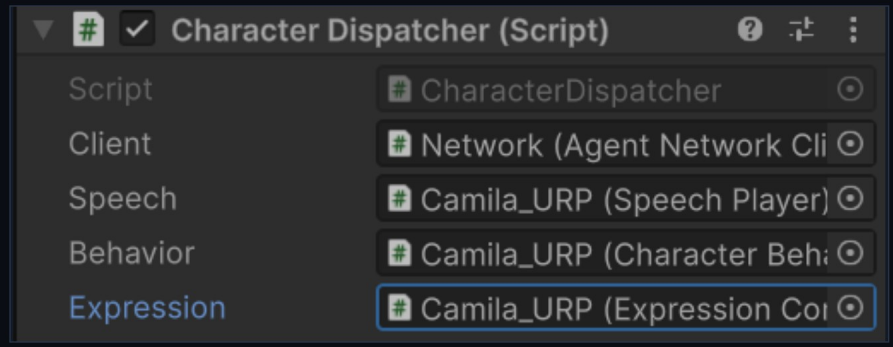
Wire the body to the dispatcher

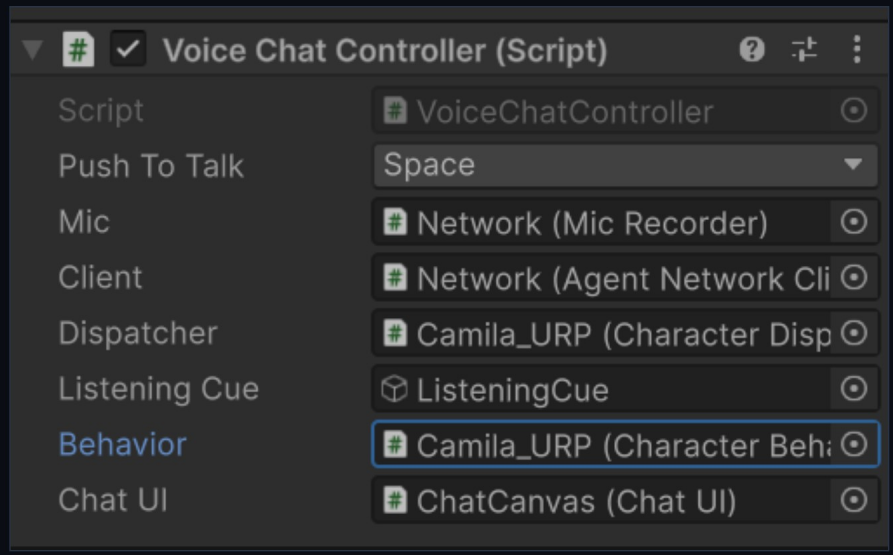
1 Select **Camila_URP** (root) → **Character Dispatcher**: Behavior = **Camila_URP**, Expression = **Camila_URP**

The two fields left empty on Day 2: drag **Camila_URP** from the Hierarchy onto each

2 Select **Network** → **Voice Chat Controller**: Behavior = **Camila_URP**

Behavior on the voice controller: **THINKING** while waiting, a **sad face** on failure





TURN-7: face and gesture start with the voice

```
public IEnumerator Handle(ChatResponse r)
{
    // LIVE-5a: download the clip first
      // ===== TURN-7 · Brain ▸ Body =====
      // set the emotion (expression) and fire the gesture
      // (behavior) from r; both may be null: check

    // LIVE-5b: speech.Play(clip)  -- the voice starts here
    if (expression != null) expression.SetEmotion("NEUTRAL");
}
```

Download **first**, then voice, gesture and face **together**

Wire: Dispatcher → **Behavior**, **Expression**; VoiceChatController → **Behavior**

Switches on THINKING while waiting and a sad face on failure

TURN-7: solution

CharacterDispatcher.cs

```
public IEnumerator Handle(ChatResponse r)
{
    // LIVE-5a: download the clip first
    // ===== TURN-7 · Brain ▸ Body =====
      if (expression != null) expression.SetEmotion(r.emotion);
      if (behavior != null) behavior.PlayGesture(r.action);

    // LIVE-5b: speech.Play(clip)  -- the voice starts here
    if (expression != null) expression.SetEmotion("NEUTRAL");
}
```

Breakout: test the full turn

Symptom	Cause and fix
Always NONE / NEUTRAL	think still returns text only: switch the seam to think_structured
Always NONE / NEUTRAL, the speech is JSON	LIVE-8 sends TEXT_PROMPT or skips _parse: use SYSTEM_PROMPT, return _parse(…)
“Input should be a valid string” on speech_text	think wraps the result in a second BrainOutput: return think_structured(…) as it is
“Extra inputs are not permitted”	TURN-5 missing: the model sent action and emotion
Always the apology	Validation fails: read the log
Gesture never plays	Trigger names differ (Waving, Thinking), or TURN-7 / wiring missing
Face flickers while talking	A preset uses a viseme (V_) shape
Waves before the voice	TURN-7 lines outside their block

What works now: words, gestures and emotions



✓ **Gestures**

WAVE, THINKING, chosen by the model

✓ **Emotions**

The face matches the reply

03

Tests, tuning and limits

How do we make it behave better?

What can it still not do?



Tune the prompt as needed

Tune: few-shot examples at the end of the prompt

```
EXAMPLES
User: Hi there!
{"speech_text": "Hi! Great to see you.", "action": "WAVE", ...}
User: My dog died yesterday.
{"speech_text": "I'm so sorry. That must be really hard.",
 "action": "NONE", "emotion": "SAD", ...}
User: Do you know what the stock market will do tomorrow?
{"speech_text": "I really can't say...", "action": "NONE", ...}
```

LLM Sampling and temperature

For each next token the LLM gives every candidate a **probability**; **sampling** draws one, weighted by those probabilities

T **Temperature** reshapes them before the draw: **low** = sharp, the top choice almost every time; **high** = flat, more variety and more mistakes

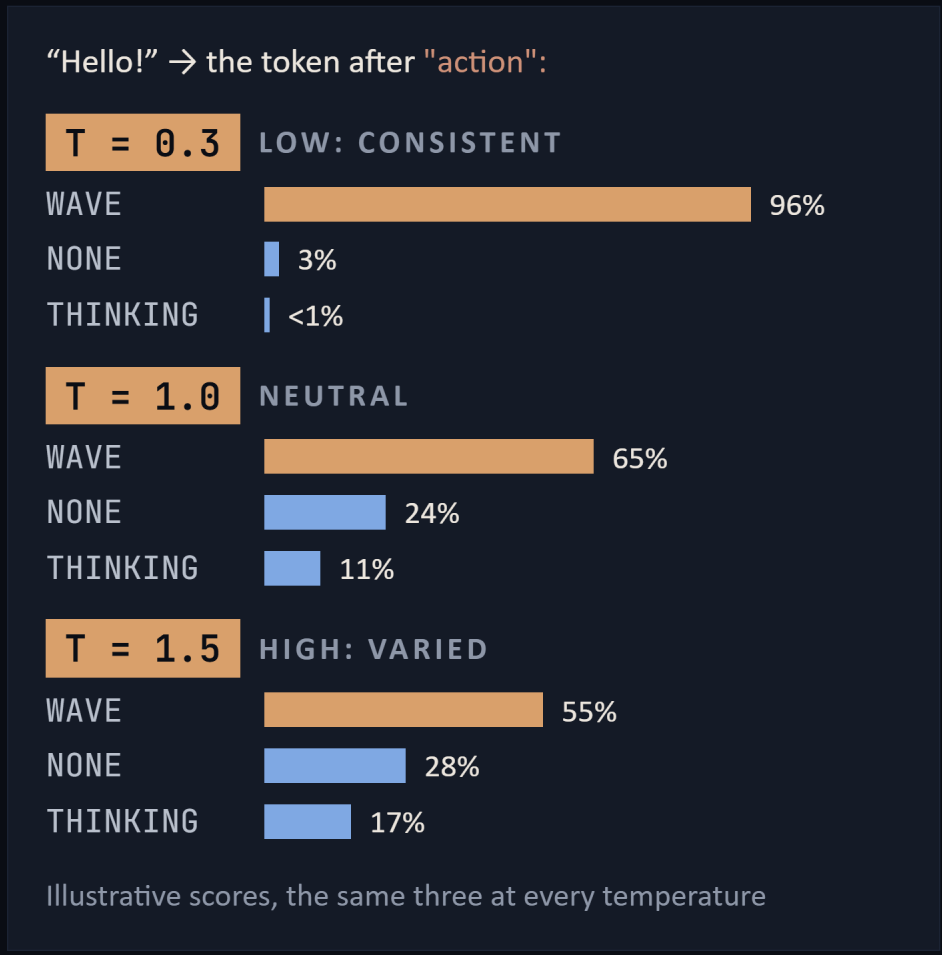
P **Top-p / top-k** also exist: only the most likely tokens enter the draw

IN OUR CODE · brain.py

```
options={"temperature": _TEMPERATURE}
```

LLM_TEMPERATURE in .env, default **0.3**

Closed lists (actions, emotions): keep it low, 0.2–0.4



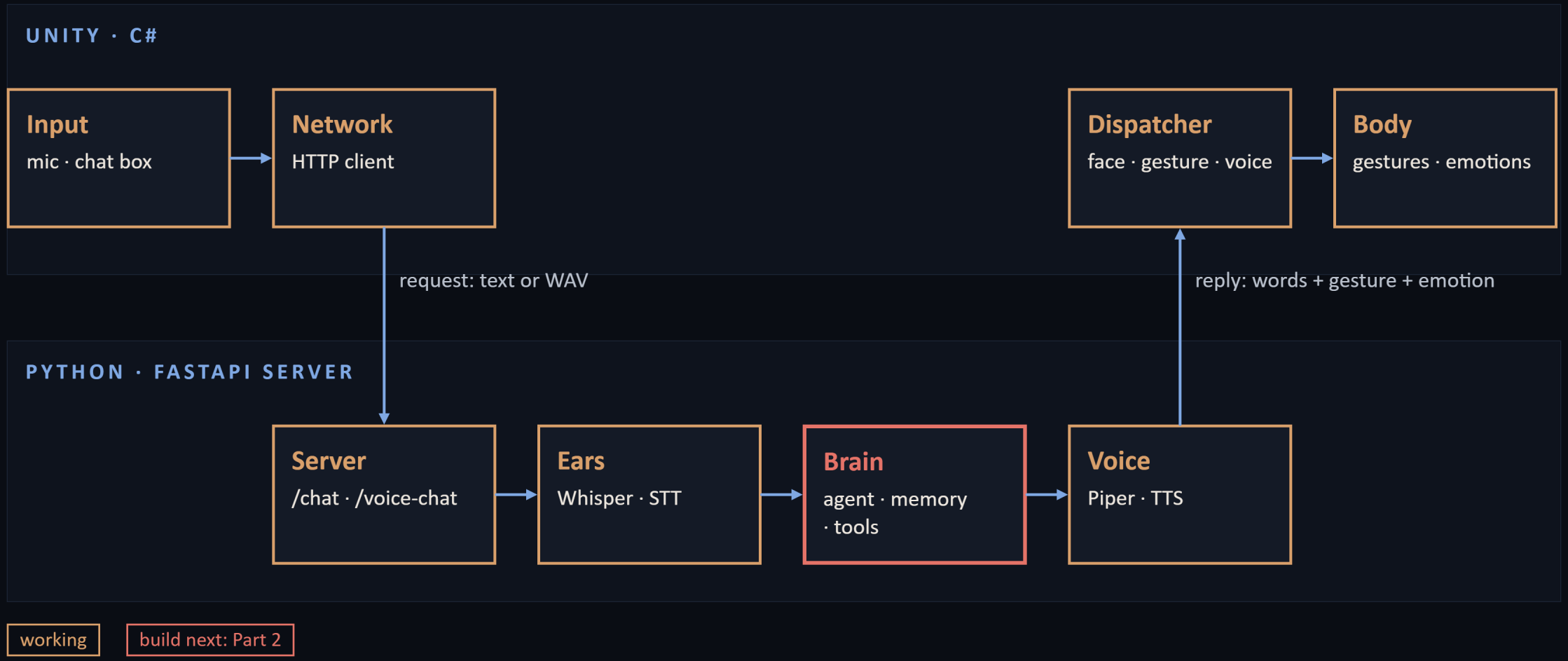
Current Limits

~~Memory~~	~~State~~	~~Reliable structure~~	~~External data~~
forgets your name between turns	no goals, no mood that persists	JSON can break	cannot check the weather

→ **Part 2**: from psychology to LLMs to LangGraph, in Part 2 with Alessandro

Unity: the body → **FastAPI: the bridge** → **The brain is whatever you plug in**

Next: the brain grows



TASK

Task: two gestures, four emotions

2 CONTEXTUAL GESTURES

Your choice: any two gestures, each with a clear **when**

Hint: Download them from mixamo.com

THE OTHER 4 BASIC EMOTIONS

`SURPRISED` `ANGRY` `AFRAID` `DISGUSTED`

Hint: what each emotion does to the face, facial expressions of emotion

Where	+2 gestures: see slides	+4 emotions: see slides
VH_Controller → Gesture layer	Day 1: 70–71 · Webinar: 8	—
CharacterBehaviorManager.Gestures.cs	Day 1: 72 · Webinar: 9	—
ExpressionController.cs → Presets	—	Day 1: 48–49 · Webinar: 13–14
schemas.py	Day 2: 28 · Webinar: 19	Day 2: 28 · Webinar: 19
brain.py → SYSTEM_PROMPT	Day 2: 8 · Webinar: 21	Day 2: 8 · Webinar: 21

AI-powered Virtual Humans

Part 1: Virtual Humans Foundations

Thank you

Questions?

Stefano Calzolari · Thu 29 October 2026

AI-Powered Virtual Humans · Politecnico di Torino

